# Plan de tests de sécurité dynamique (DAST) — OWASP ZAP — FileFlow

| | |
|---|---|
| **Titre** | Plan de tests de sécurité dynamique (DAST) — OWASP ZAP — Plateforme FileFlow |
| **Version** | 1.1 |
| **Date** | 27 octobre 2026 |
| **Statut** | Validé |
| **Propriétaire** | Driss OUAAZIZ — Software Architect |
| **Classification** | Interne |

> Références méthodologiques : OWASP *Web Security Testing Guide* (WSTG) v4.2 ; OWASP Top 10 (2021) ; OWASP ZAP — *Automation Framework* et *API Scan* ; OWASP *DevSecOps Guideline*.

## Table des matières


1. Objet et périmètre
2. Contexte et état actuel
3. Objectifs des tests
4. Référentiel méthodologique
5. Environnement de test
6. Jeu de données de test (amorçage)
7. Stratégie de test et types de scans
8. Prise en compte des spécificités de FileFlow
9. Politique de scan et couverture des règles
10. Gestion des faux positifs
11. Critères d'acceptation (porte de sécurité)
12. Automatisation et intégration continue
13. Livrables et reporting
14. Phasage et planning
15. Rôles et responsabilités
16. Risques de la campagne et mitigations
17. Références

- Annexe A — Plan d'automatisation ZAP (Automation Framework)
- Annexe B — Amorçage des données de test
- Annexe C — Journal de triage des alertes (gabarit)


## 1. Objet et périmètre

Ce document définit la stratégie, l'environnement, la configuration et les critères d'acceptation des tests de sécurité **dynamiques** (DAST — *Dynamic Application Security Testing*) de la plateforme **FileFlow**, conduits avec

**OWASP ZAP**.

Il complète l'analyse statique et l'analyse de dépendances (voir le *Dossier de sécurité et modélisation des menaces*, `06-dossier-securite-threat-model.md`) en exerçant l'application **en cours d'exécution**, telle qu'un attaquant la voit.

**Dans le périmètre :**

- L'API REST sous `/api` (surface authentifiée : fichiers, partages, uploads, favoris, collaboration, administration).
- Les points d'entrée publics non authentifiés : liens de partage (`/api/s/{token}`), demandes d'upload (`/api/r/{token}`), sonde de santé (`/api/health`).
- L'application web (SPA) servie par le backend, y compris les en-têtes de sécurité et la gestion des cookies de session.
- Les flux d'authentification et de gestion de session (connexion, 2FA, révocation).

**Hors périmètre :**

- La terminaison TLS et la configuration du reverse-proxy (testées au niveau de l'infrastructure, en amont).
- Les vulnérabilités de code source et de dépendances (couvertes par le SAST, `cargo-audit`, `trivy`, `gitleaks` — voir document 06).
- Les tests de charge et de performance.

## 2. Contexte et état actuel

Une **campagne initiale de base (baseline, passive et non authentifiée)** a déjà été exécutée contre une instance en fonctionnement. Résultat :

| INDICATEUR | VALEUR |
|---|---|
| **Alertes en échec (FAIL)** | 0 |
| **Avertissements (WARN)** | 5 (durcissement d'en-têtes, faible criticité) |
| **Contrôles réussis (PASS)** | 62 |
| **URL explorées** | 6 (coquille du SPA, ressources statiques, `/api/health`) |

**Limite identifiée.** FileFlow étant une application monopage (SPA) à session par cookie, l'exploration passive n'a atteint que la **surface publique**. La surface **authentifiée** — c'est-à-dire l'essentiel de la logique métier et du risque — n'a **pas** été testée dynamiquement. Les avertissements résiduels (en-tête `Permissions-Policy` absent, `COOP` / `COEP` non positionnés, `style-src 'unsafe-inline'`, ressources statiques cachables) sont documentés comme risque accepté / durcissement optionnel dans le document 06.

**Évolutions depuis la baseline (octobre 2026).** Le rapport d'audit d'octobre 2026 (document 14) a conduit à des correctifs qui modifient le comportement attendu de plusieurs points d'entrée, et donc la configuration et l'interprétation de la campagne (détail au §8) :

- Chemins virtuels normalisés avant tout contrôle d'accès : `.` et barres multiples supprimés, tout segment `..` rejeté (400). Un chemin contenant `..` qui obtient autre chose qu'un 400 est une anomalie.
- Limitation des tentatives de mot de passe sur `/api/s/{token}` et `/api/r/{token}` (5 échecs par lien ou par IP → réponse **429** pendant 15 minutes).
- Dépôt public : le champ mot de passe doit **précéder** les fichiers dans le formulaire multipart ; taille plafonnée par `PUBLIC_UPLOAD_MAX_MB` (100 Mo par défaut) ; créer une demande de dépôt exige

l'écriture propre sur le dossier ; un dépôt n'écrase jamais un fichier existant.

- Liens à téléchargement limité : plafond atomique, en-tête `Range` ignoré (fichier entier servi et compté) ; liens publics et demandes de dépôt rattachés au cycle de vie de la ressource, et 404 uniforme si le créateur a perdu l'accès.
- Connexion : verrouillage par IP, par couple (utilisateur, IP) et par compte (50 échecs toutes IP confondues).
- `POST /api/auth/logout` répond 200 sans authentification et efface toujours les cookies ; désactiver la 2FA ou régénérer les codes exige mot de passe **et** code.
- Le dernier administrateur actif ne peut être ni rétrogradé, ni désactivé, ni supprimé (400).

Le présent plan organise la **suite** : couverture authentifiée, scan actif et pilotage par la spécification d'API.

## 3. Objectifs des tests

| RÉF. | OBJECTIF |
|---|---|
| G1 | Confirmer l'absence de vulnérabilités d'injection (SQL, commande, en-tête, chemin) sur l'API en fonctionnement |
| G2 | Vérifier le contrôle d'accès en conditions réelles : absence d'accès horizontal/vertical non autorisé (IDOR, élévation de privilèges) |
| G3 | Valider la robustesse de l'authentification et de la gestion de session (cookies, révocation, expiration) |
| G4 | Confirmer la présence et la correcte configuration des en-têtes de sécurité et de la CSP sur l'ensemble des réponses |
| G5 | Couvrir dynamiquement **l'intégralité** des points d'entrée de l'API à partir de la spécification OpenAPI |
| G6 | Intégrer la campagne à un processus **répétable et automatisable** (non-régression à chaque version) |

## 4. Référentiel méthodologique

Les tests suivent la méthodologie de l'**OWASP Web Security Testing Guide (WSTG)**, en priorisant les catégories pertinentes pour FileFlow :

| CATÉGORIE WSTG | APPLICATION À FILEFLOW |
|---|---|
| **Gestion des sessions (WSTG-SESS)** | Cookies `HttpOnly` / `SameSite=Strict`, révocation par version de jeton, expiration |
| **Authentification (WSTG-ATHN)** | Connexion, 2FA TOTP, verrouillage anti-force brute, égalisation temporelle |
| **Autorisation (WSTG-ATHZ)** | RBAC, restriction par chemins, IDOR sur jetons et identifiants |
| **Validation des entrées (WSTG-INPV)** | Injection, traversée de répertoires, upload de fichiers |
| **Configuration (WSTG-CONF)** | En-têtes de sécurité, méthodes HTTP, gestion des erreurs |
| **Logique métier (WSTG-BUSL)** | Quotas, limites de téléchargement, expiration des liens |

La cotation des alertes s'appuie sur les niveaux de risque ZAP (Informational, Low, Medium, High) et la correspondance **OWASP Top 10 (2021)**.

## 5. Environnement de test

Les tests sont exécutés **exclusivement contre une instance dédiée et jetable**, jamais contre un environnement de production ni contre des données réelles.

| PARAMÈTRE | VALEUR | JUSTIFICATION |
|---|---|---|
| **Image** | `fileflow` (build statique musl / distroless) | Parité avec la production |
| **Données** | Volumes `data/` et `config/` **jetables**, réinitialisés à chaque campagne | Isolation, reproductibilité |
| `COOKIE_SECURE` | `false` | L'instance de test est servie en HTTP local (pas de TLS) |
| `TRUST_PROXY_HEADERS` | `false` | Pas de proxy en amont sur l'instance de test |
| **Réseau** | Hôte isolé, port local non exposé publiquement | Éviter toute fuite ou impact externe |
| **Cycle de vie** | Conteneur créé avant la campagne, **détruit et reconstruit après** | Les scans actifs sont destructifs sur les données |

**Principe.** Parce que le scan actif exerce les points d'entrée destructifs (suppression, corbeille, révocation, gestion des comptes), l'instance est considérée comme **sacrifiable** : aucune donnée n'y a de valeur et elle est recréée à neuf après chaque exécution.

## 6. Jeu de données de test (amorçage)

Pour exercer la surface authentifiée et les points d'entrée à jeton, l'instance est amorcée avec un jeu de données déterministe (voir Annexe B) :

- Un compte **administrateur** (sans 2FA, pour permettre l'authentification automatisée).
- Trois comptes aux rôles distincts — **Editor**, **Viewer**, **User** — pour tester l'autorisation horizontale et verticale (objectif G2).
- Un **groupe** avec `allowed_paths` restreints, pour tester le confinement par chemin.
- Une arborescence de **fichiers et dossiers** de test.
- Un **lien de partage public** (avec et sans mot de passe) et une **demande d'upload** publique — pour fournir des jetons valides aux points d'entrée `/s/{token}` et `/r/{token}`.
- Un **partage interne** utilisateur-à-utilisateur, pour tester le contrôle d'accès transitif.

Les jetons générés à l'amorçage sont injectés dans la configuration ZAP afin que l'explorateur et le scanner actif atteignent les points d'entrée publics.

## 7. Stratégie de test et types de scans

La campagne combine plusieurs modes ZAP, du moins au plus intrusif.

### 7.1 Scan passif (baseline)

Analyse non intrusive du trafic : en-têtes de sécurité, attributs de cookies, fuites d'information, contenu cachable. Déjà exécuté (§2) ; rejoué à chaque campagne comme socle.

### 7.2 Exploration (spider)

- **Spider traditionnel** : découverte des ressources liées (coquille du SPA, ressources statiques).
- **Spider AJAX** : exploration pilotée par navigateur pour suivre les routes rendues côté client par le SPA. Nécessaire car le contenu applicatif est rendu en JavaScript.

L'exploration seule ne découvre pas les points d'entrée REST (non liés dans le HTML) : elle est **complétée** par l'import de la spécification OpenAPI (§7.4).

### 7.3 Scan actif

Envoi de charges d'attaque sur les points d'entrée découverts : injection SQL, injection de commande, traversée de répertoires, XSS réfléchi, manipulation de paramètres, injection d'en-têtes. C'est le cœur de la détection de vulnérabilités exploitables (objectifs G1, G2).

### 7.4 Scan piloté par l'API (OpenAPI)

FileFlow expose un contrat **OpenAPI 3.1** complet ( `08-api/openapi.yaml` , 56 chemins). Ce contrat est **importé dans ZAP** ( `zap-api-scan.py` ou tâche `openapi` de l'Automation Framework) afin que le scanner actif couvre **toutes** les opérations de l'API, y compris celles que l'exploration ne peut pas découvrir. C'est le levier principal de couverture (objectif G5).

### 7.5 Scan authentifié

Sans authentification, seule la surface publique est testée. La campagne configure donc :

- **Méthode d'authentification** : connexion par requête JSON sur `POST /api/auth/login` ( `{ "username", "password" }` ), qui dépose le cookie de session `fileflow_token` .
- **Gestion de session** : basée sur les cookies ( `cookieBasedSessionManagement` ).
- **Vérification de session** : appel périodique de `GET /api/auth/me` (indicateur de session valide / expirée) pour la reconnexion automatique.
- **Utilisateurs** : administrateur + comptes Editor/Viewer/User, permettant de rejouer le scan sous plusieurs rôles et de détecter les défaillances de contrôle d'accès.

## 8. Prise en compte des spécificités de FileFlow

| SPÉCIFICITÉ | IMPACT SUR LA CAMPAGNE | TRAITEMENT |
|---|---|---|
| **Limitation anti-force brute — authentification ( `login` : 5 échecs par IP ou par couple utilisateur/IP, 50 échecs par compte toutes IP → verrou 15 min ; `2FA` : 5 échecs par utilisateur, y compris pour `disable` et `regenerate` )** | Le fuzzing de la connexion déclenchera le verrou (comportement attendu, réponse 429) ; l'API authentifiée n'est **pas** limitée | Exclure `/auth/login` et `/auth/2fa/*` du scan actif automatisé ; tester le verrouillage par un cas dédié. Les verrous étant en mémoire, redémarrer l'instance pour les remettre à zéro |
| **Limitation anti-force brute — liens publics ( `/s/{token}` , `/r/{token}` protégés : 5 échecs par lien ou par IP → 429 pendant 15 min)** | Le scan actif du formulaire de mot de passe verrouille le lien **et** l'adresse IP du scanner : toutes les réponses suivantes sur les liens protégés deviennent 429 | Scanner activement un lien et une demande de dépôt **sans** mot de passe ; réserver les variantes protégées à un cas dédié (429 attendu à la sixième tentative). Une réponse 429 sur ces routes n'est pas une anomalie |
| **Liens à téléchargement limité (plafond atomique, en-tête `Range` ignoré)** | Un lien plafonné est épuisé dès les premières requêtes du scanner (404 ensuite) | Lien **sans plafond** pour le scan actif ; cas dédié : téléchargements parallèles d'un lien à usage unique (un seul succès) et requête `Range` sur un lien plafonné (réponse 200 complète, comptée) puis sur un lien non plafonné (206) |

| SPÉCIFICITÉ | IMPACT SUR LA CAMPAGNE | TRAITEMENT |
|---|---|---|
| **Cycle de vie des liens** (liens et demandes de dépôt suivent renommage et déplacement, disparaissent à la suppression ; droits du créateur revérifiés) | Le scan actif des routes `rename`, `move`, `delete` peut invalider les jetons d'amorçage | Placer les ressources des liens hors de l'arborescence exposée au scan actif, ou réamorcer les jetons ; une réponse 404 après suppression ou retrait de droits du créateur est le comportement attendu |
| **Endpoints destructifs** (`delete`, `move`, `trash/empty`, `admin/users` DELETE, `revoke-sessions`, `logout-all`) | Peuvent détruire les données de test ou **invalider la session du scanner** (`logout` efface désormais toujours les cookies, avec ou sans session valide) | Instance jetable ; **exclure** `logout`, `logout-all`, `revoke-sessions` et la suppression du compte administrateur du scan actif (voir périmètre ci-dessous). Le dernier administrateur actif est protégé (400 sur rétrogradation, désactivation ou suppression) : une réponse 400 sur ces opérations est attendue |
| **Protection CSRF par `SameSite=Strict` + absence de CORS** | Pas de jeton anti-CSRF applicatif | Documenter comme contrôle en place ; désactiver la gestion de jeton anti-CSRF côté ZAP |
| **2FA (TOTP)** | Un code temporel bloque l'authentification automatisée | Comptes de test **sans 2FA** pour l'automatisation ; le flux 2FA fait l'objet d'un test manuel dédié |
| **Uploads** (multipart + chunké) | Points d'entrée sensibles (nom de fichier, extension, quota) | Inclure ; cas de test dédiés : extension bloquée, nom de fichier piégé (traversée, caractères de contrôle), dépassement de quota |
| **Dépôt public** (`/r/{token}`) | Mot de passe exigé **avant** les fichiers dans le multipart ; plafond `PUBLIC_UPLOAD_MAX_MB` ; jamais d'écrasement | Cas dédiés : fichier envoyé avant le mot de passe (refusé, aucun fichier écrit), dépassement du plafond, dépôt d'un nom existant (fichier numéroté `nom (1).ext`), demande de dépôt créée par un compte en lecture seule (refusée) |
| **Points d'entrée publics à jeton** (`/s/{token}`, `/r/{token}`) | Inaccessibles sans jeton valide | Jetons injectés depuis l'amorçage (§6) ; tester l'injection sur le formulaire de mot de passe et l'upload public |

**Périmètre d'inclusion / exclusion du scan actif :**

- **Inclus** : `/api/files/*`, `/api/uploads/*`, `/api/trash*`, `/api/versions*`, `/api/metadata/*`, `/api/shares*`, `/api/shared-folders*`, `/api/internal-shares*`, `/api/favorites*`, `/api/upload-requests*`, `/api/s/{token}`, `/api/r/{token}`, `/api/admin/groups*`, `/api/admin/audit*`.
- **Exclus** (auto-destruction de session ou de compte) : `/api/auth/logout`, `/api/auth/logout-all`, `/api/admin/users/*/revoke-sessions`, `DELETE /api/admin/users/{username}`, ainsi que `/api/auth/login` et `/api/auth/2fa/*` du fuzzing automatisé (testés par cas dédiés).

## 9. Politique de scan et couverture des règles

- **Seuil d'alerte** : niveau *Medium* (réduit le bruit tout en conservant les vulnérabilités significatives) ; une passe *Low* est réalisée pour le durcissement.

- **Force d'attaque** : *Medium* par défaut, *High* sur les points d'entrée d'injection et de traversée de chemins.
- **Règles actives prioritaires** : injection SQL, injection de commande OS, traversée de répertoires / inclusion de fichiers, XSS (réfléchi et persistant), injection d'en-têtes, désérialisation, contournement d'autorisation.
- **Règles passives prioritaires** : en-têtes de sécurité, attributs de cookies, fuites d'information, divulgation d'erreurs.

## 10. Gestion des faux positifs

- Les faux positifs et risques acceptés sont consignés dans un **journal de triage** (Annexe C) et neutralisés par des **filtres d'alerte ZAP** (par identifiant de règle et URL), afin que les campagnes suivantes restent lisibles.
- Les 5 avertissements de la baseline (§2) sont pré-qualifiés : `style-src 'unsafe-inline'` (accepté), en-têtes `Permissions-Policy` / `COOP` / `COEP` (durcissement recommandé, suivi dans le registre de dette `12-…`), contenu statique cachable (bénin).
- Tout nouveau faux positif est justifié par écrit avant d'être filtré.

## 11. Critères d'acceptation (porte de sécurité)

| SÉVÉRITÉ | CRITÈRE DE SUCCÈS |
|---|---|
| **High / Critical** | **Zéro** alerte non résolue. Bloquant pour la mise en production |
| **Medium** | Chaque alerte triée : corrigée, ou justifiée et acceptée par écrit |
| **Low / Informational** | Consignées ; traitées dans le cadre du durcissement continu |

La campagne est **réussie** lorsque : aucune alerte High/Critical ne subsiste, toute alerte Medium est tracée avec une décision, et le rapport est archivé.

## 12. Automatisation et intégration continue

La campagne est décrite comme **plan d'automatisation ZAP** (*Automation Framework*, fichier YAML versionné — Annexe A), garantissant sa reproductibilité.

- **Déclenchement** : sur une base **planifiée** (hebdomadaire) et à la demande — et non à chaque *pull request*, en raison de la durée du scan actif et du besoin d'une instance en fonctionnement.
- **Intégration** : ajout d'une tâche dédiée au workflow de sécurité existant (`.github/workflows/security.yml`), qui construit l'image, démarre l'instance jetable, l'amorce (Annexe B), exécute le plan ZAP puis publie le rapport.
- **Verdict** : la tâche échoue si la porte de sécurité (§11) n'est pas respectée (présence d'une alerte High/Critical).

## 13. Livrables et reporting

| LIVRABLE | FORMAT | USAGE |
|---|---|---|
| **Rapport de campagne** | HTML + JSON + Markdown | Lecture humaine et archivage |
| **Export SARIF** | SARIF | Intégration à la revue de code / tableau de bord de sécurité |
| **Journal de triage** | Tableau (Annexe C) | Suivi des décisions sur les alertes |

| LIVRABLE | FORMAT | USAGE |
|---|---|---|
| **Plan d'automatisation** | YAML | Rejouabilité et versionnement |

Les rapports sont horodatés et conservés pour assurer la traçabilité et la comparaison entre versions.

## 14. Phasage et planning

| PHASE | CONTENU | ÉTAT |
|---|---|---|
| **P1 — Baseline** | Scan passif non authentifié | **Réalisée** (§2) |
| **P2 — Authentifié + API** | Contexte, authentification, import OpenAPI, scan actif sous plusieurs rôles | À réaliser |
| **P3 — Automatisation** | Plan Automation Framework, amorçage scripté, intégration CI | À réaliser |
| **P4 — Récurrence** | Exécution planifiée et suivi des tendances entre versions | Continu |

## 15. Rôles et responsabilités

| RÔLE | RESPONSABILITÉ |
|---|---|
| **Software Architect (Driss OUAAZIZ)** | Définition du plan, configuration ZAP, exécution, triage, décision de la porte de sécurité |
| **Exploitation** | Mise à disposition de l'instance jetable et des ressources d'exécution |

## 16. Risques de la campagne et mitigations

| RÉF. | RISQUE | MITIGATION |
|---|---|---|
| RC1 | Corruption / perte de données par le scan actif | Instance et données **jetables**, reconstruites après chaque campagne |
| RC2 | Invalidation de la session du scanner en cours de scan | Exclusion de `logout`, `logout-all`, `revoke-sessions` du scan actif |
| RC3 | Déclenchement du verrou anti-force brute faussant les résultats (connexion, 2FA, liens publics protégés) | Exclusion des points d'authentification du fuzzing ; liens publics sans mot de passe pour le scan actif ; cas de test dédiés ; redémarrage de l'instance pour purger les verrous en mémoire |
| RC4 | Faux sentiment de sécurité (couverture partielle) | Pilotage par OpenAPI pour la complétude ; documentation explicite du périmètre |
| RC5 | Impact hors instance de test | Réseau isolé, port non exposé, jamais contre la production |

## 17. Références


- OWASP — *Web Security Testing Guide* (WSTG) v4.2.
- OWASP — *Top 10* (2021).
- OWASP ZAP — *Automation Framework* ; *API Scan* (`zap-api-scan.py`) ; *Baseline Scan*.
- OWASP — *DevSecOps Guideline*.
- Dossier de sécurité et modélisation des menaces — `06-dossier-securite-threat-model.md`.
- Documentation de l'API — `08-api/openapi.yaml`.

## Annexe A — Plan d'automatisation ZAP (Automation Framework)

Plan à adapter (identifiants, URL de base, jetons d'amorçage). Exécution : `zap.sh -cmd -autorun zap-plan.yaml`.

```yaml
env:
  contexts:
    - name: fileflow
      urls:
        - http://127.0.0.1:18080
      includePaths:
        - "http://127.0.0.1:18080/api/.*"
      excludePaths:
        - "http://127.0.0.1:18080/api/auth/logout"
        - "http://127.0.0.1:18080/api/auth/logout-all"
        - "http://127.0.0.1:18080/api/auth/login"
        - "http://127.0.0.1:18080/api/auth/2fa/.*"
        - "http://127.0.0.1:18080/api/admin/users/.*/revoke-sessions"
      authentication:
        method: json
        parameters:
          loginPageUrl: http://127.0.0.1:18080/api/auth/login
          loginRequestUrl: http://127.0.0.1:18080/api/auth/login
          loginRequestBody: '{"username":"{%username%}","password":"{%password%}"}'
        verification:
          method: poll
          pollUrl: http://127.0.0.1:18080/api/auth/me
          pollFrequency: 60
          pollUnits: requests
          loggedInRegex: '"username"'
          loggedOutRegex: '"error"'
      sessionManagement:
        method: cookie
      users:
        - name: admin
          credentials:
            username: admin
            password: "${ADMIN_PASSWORD}"
        - name: editor
          credentials:
            username: editor_test
            password: "${TEST_PASSWORD}"
  parameters:
    failOnError: true
    progressToStdout: true

jobs:
  - type: openapi               # import du contrat pour la couverture complète
    parameters:
      apiFile: /zap/wrk/openapi.yaml
      targetUrl: http://127.0.0.1:18080/api
      context: fileflow

  - type: spiderAjax            # exploration du SPA rendu côté client
    parameters:
      context: fileflow
```

```yaml
        user: admin
        maxDuration: 5

    - type: passiveScan-wait
      parameters:
        maxDuration: 5

    - type: activeScan           # scan actif authentifié
      parameters:
        context: fileflow
        user: admin
        policy: Default Policy
        maxRuleDurationInMins: 5

    - type: report
      parameters:
        template: traditional-html
        reportDir: /zap/wrk
        reportFile: zap-fileflow-report
    - type: report
      parameters:
        template: sarif-json
        reportDir: /zap/wrk
        reportFile: zap-fileflow-sarif
```

## Annexe B — Amorçage des données de test

Séquence à exécuter avant le scan (illustrée avec `curl`, en réutilisant le cookie de session). L'objectif est de créer les comptes, données et **jetons** nécessaires.

```bash
BASE=http://127.0.0.1:18080/api
JAR=cookies.txt

# 1. Connexion administrateur (dépose le cookie de session)
curl -s -c $JAR -X POST $BASE/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"'"$ADMIN_PASSWORD"'"}'

# 2. Comptes de rôles distincts (autorisation horizontale/verticale)
for r in editor viewer user; do
  curl -s -b $JAR -X POST $BASE/admin/users \
    -H 'Content-Type: application/json' \
    -d '{"username":"'"${r}"'_test","password":"'"$TEST_PASSWORD"'","role":"'"$r"'"}'
done

# 3. Arborescence et fichier de test
curl -s -b $JAR -X POST $BASE/files/mkdir \
  -H 'Content-Type: application/json' -d '{"path":"/","name":"tests"}'

# 4. Lien de partage public sans mot de passe ni plafond (jeton pour /s/{token})
curl -s -b $JAR -X POST $BASE/shares \
  -H 'Content-Type: application/json' \
  -d
'{"file_path":"/tests/exemple.txt","expires_in_secs":86400,"password":null,"max_downloads":null}'
```

```
        # 4 bis. Lien à usage unique (cas dédié plafond / Range, hors scan actif)
        curl -s -b $JAR -X POST $BASE/shares \
          -H 'Content-Type: application/json' \
          -d
'{"file_path":"/tests/exemple.txt","expires_in_secs":86400,"password":null,"max_downloads":1}'

        # 5. Demande d'upload publique (fournit un jeton pour /r/{token})
        curl -s -b $JAR -X POST $BASE/upload-requests \
          -H 'Content-Type: application/json' \
          -d '{"name":"depot-test","folder_path":"/tests"}'
```

Les jetons retournés aux étapes 4 et 5 sont injectés dans le périmètre ZAP afin d'exercer les points d'entrée publics.

## Annexe C — Journal de triage des alertes (gabarit)

| RÉF. | RÈGLE ZAP (ID) | POINT D'ENTRÉE | SÉVÉRITÉ | STATUT | DÉCISION / JUSTIFICATION |
|---|---|---|---|---|---|
| A1 | | | | Faux positif / Confirmé / Accepté | |
| A2 | | | | | |